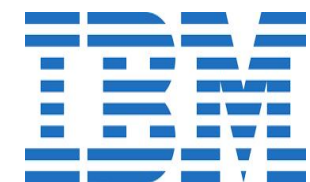


SKIP-SALSA: Skip Synchronous Fusion of ASR LLM Decoder

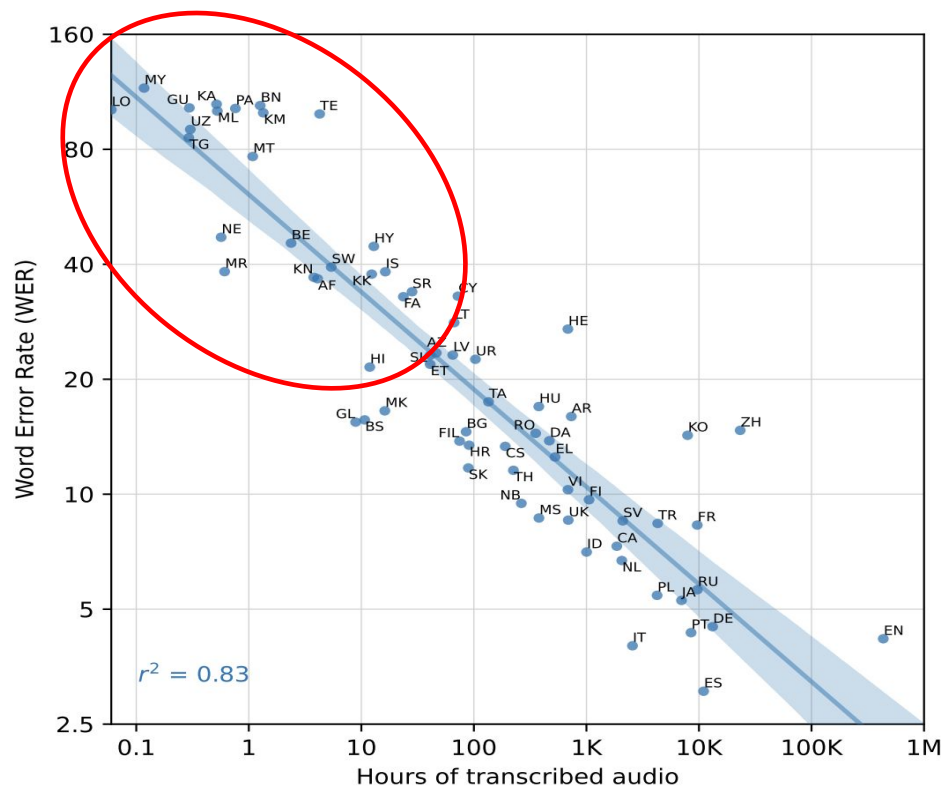
Ashish Mittal^{1,2}, Darshan Prabhu²

Sunita Sarawagi², Preethi Jyothi²

¹IBM Research, ²IIT Bombay



ASR performance



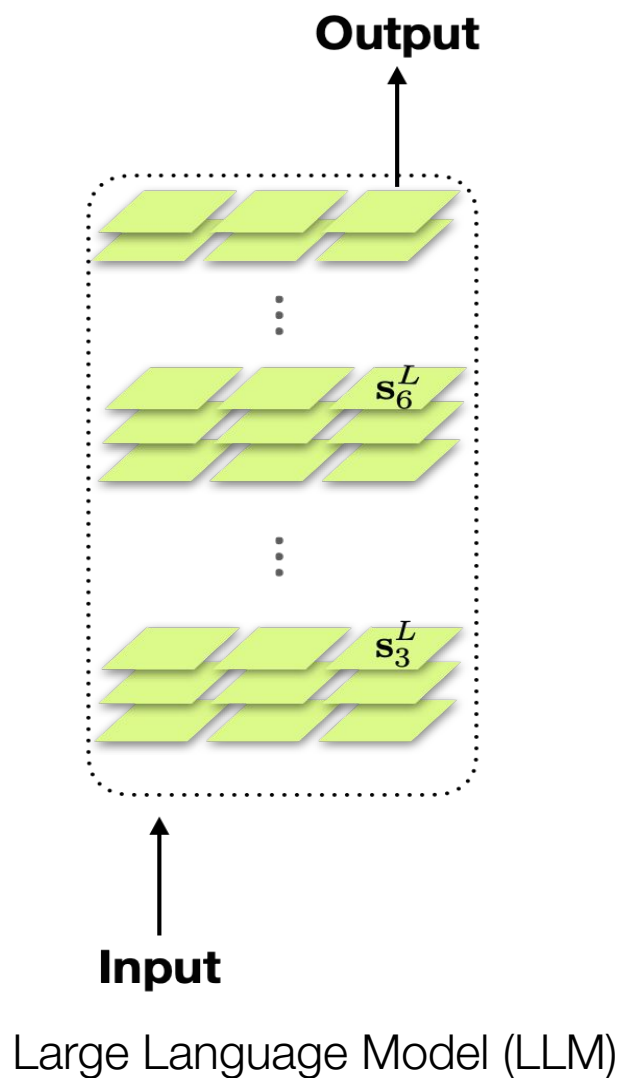
High-Resource Languages: ASR systems perform very well

Low-Resource Languages:

- Performance remains sub-optimal
- Acquiring transcribed data is still a challenge

Reference: Radford, Alec, et al. "Robust speech recognition via large-scale weak supervision."

Large Language Models

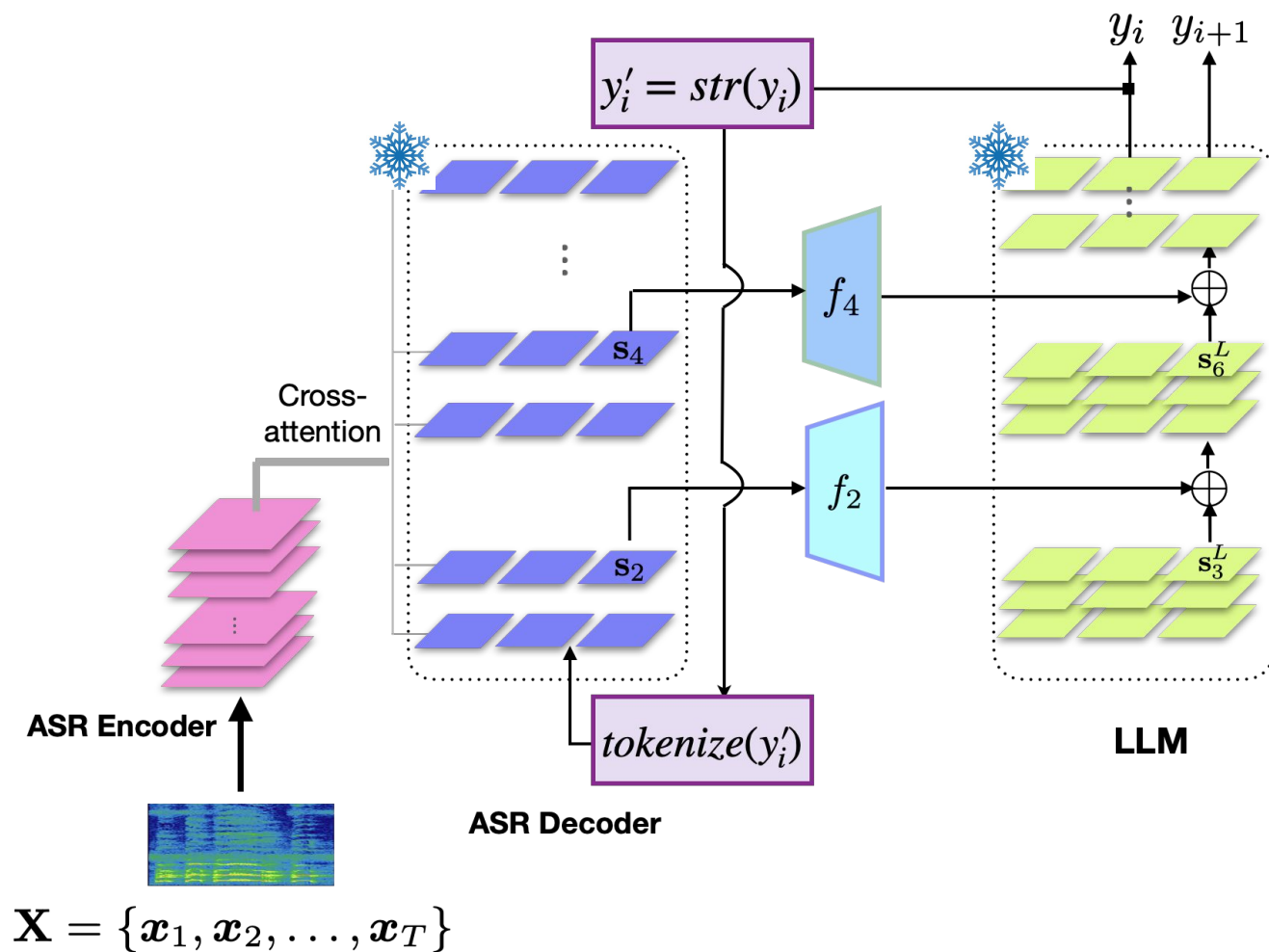


- Massive scale
- Impressive language generation
- Multilingual understanding

Prior Methods

- Prior ASR-LLM contextualization techniques are expensive
 - Prefix Tuning
 - Requires a substantial amount of paired data—often in the order of 1000 hours—to achieve reasonable performance, which is unavailable for low-resource languages.
 - Second Pass Rescoring in ASR Error correction
 - High decoding latency
 - N-best lists unreliable for low resource languages
 - LLM agnostic to audio
- We proposed a lightweight approach - **SALSA** [Interspeech 2024] that couples the decoder layers of pretrained ASR and LLM models

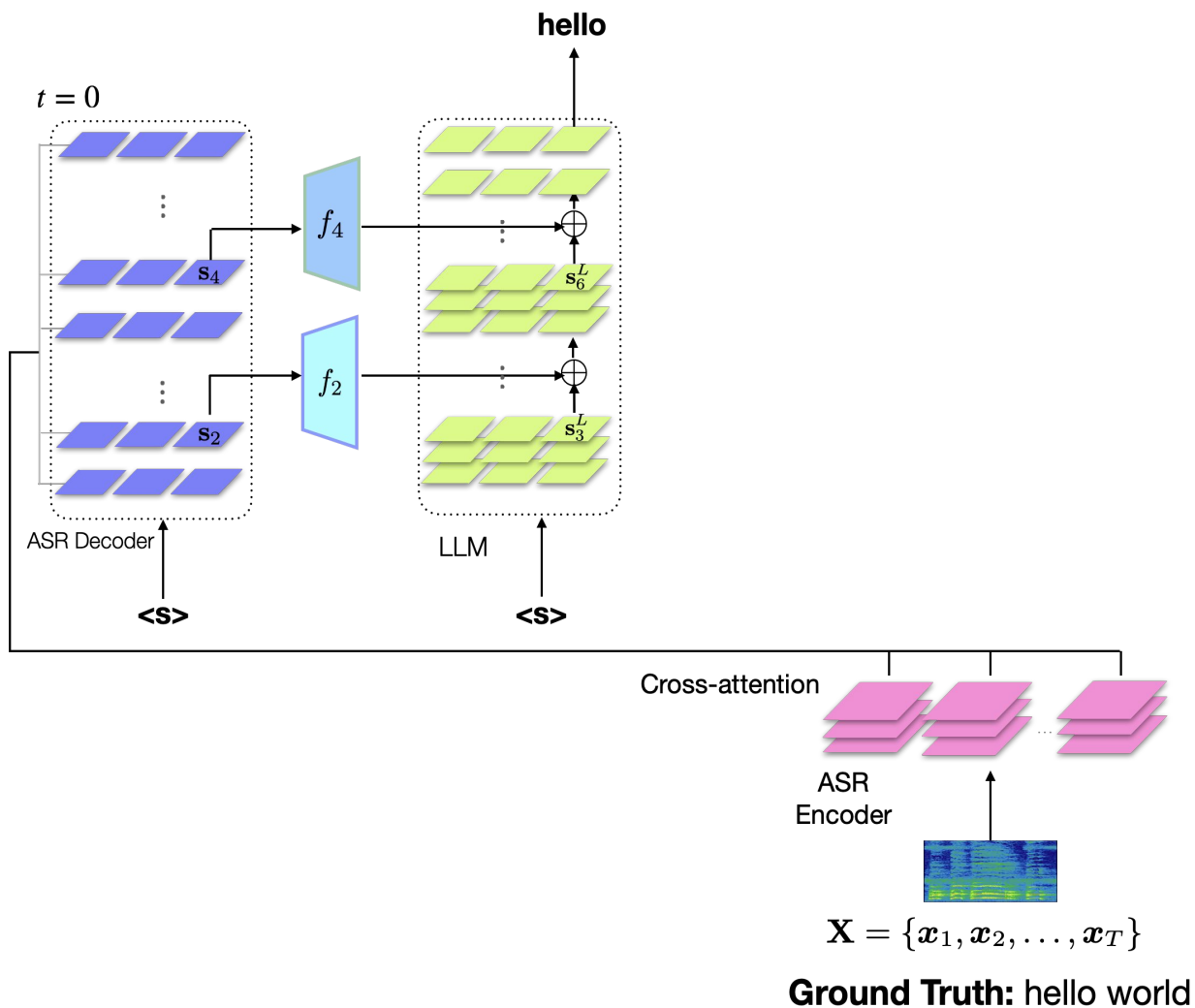
SALSA (Speedy ASR-LLM Synchronous Aggregation)



ASR-LLM Aggregation:

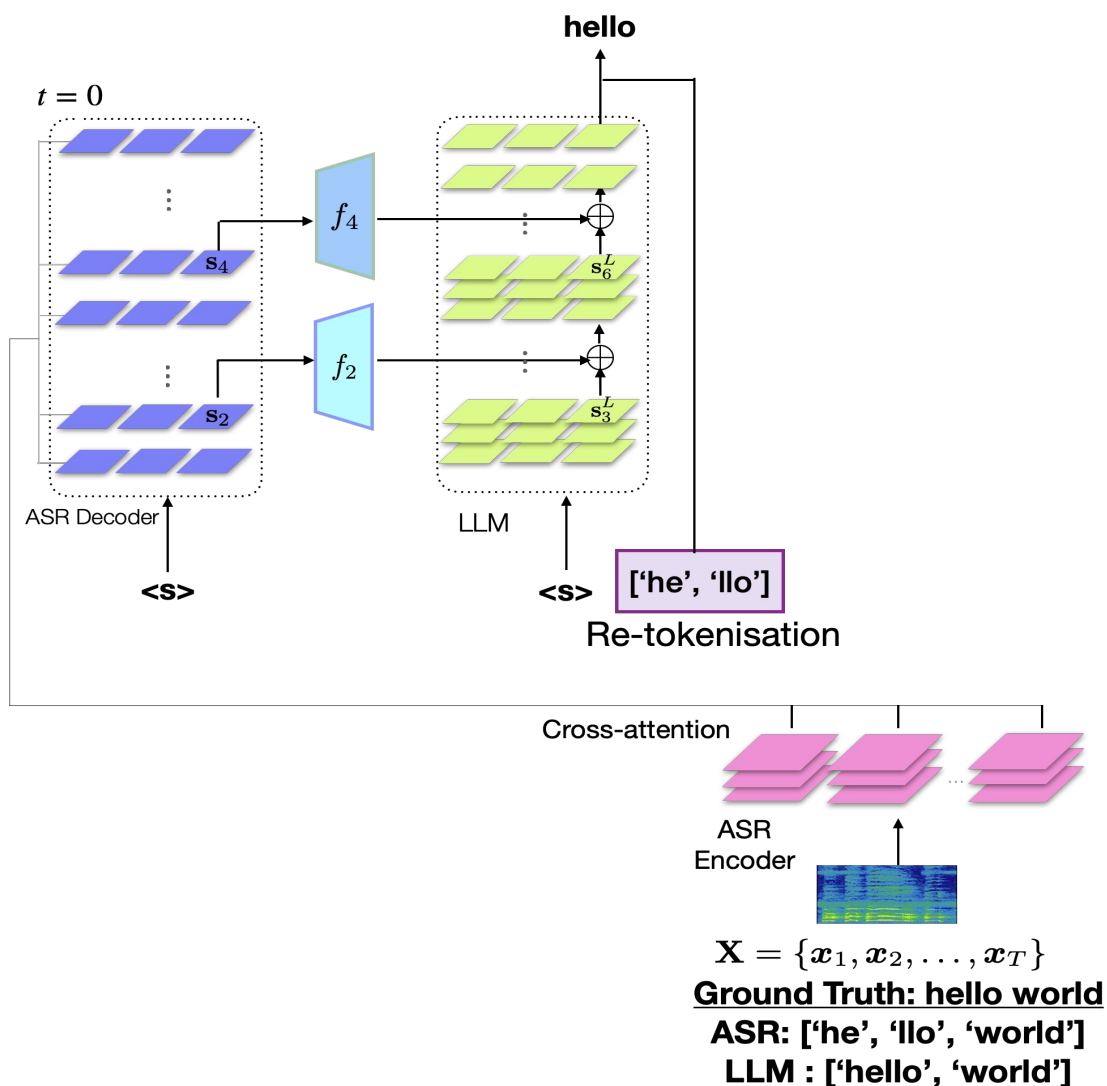
- The LLM generates the next token autoregressively
- LLM predictions are fed back to the ASR decoder (Synchronous)
- Connect ASR decoder state(s) via learnable projection layers to the LLM
- ASR and LLM modules are frozen

ASR-LLM Aggregation: (Working Example)

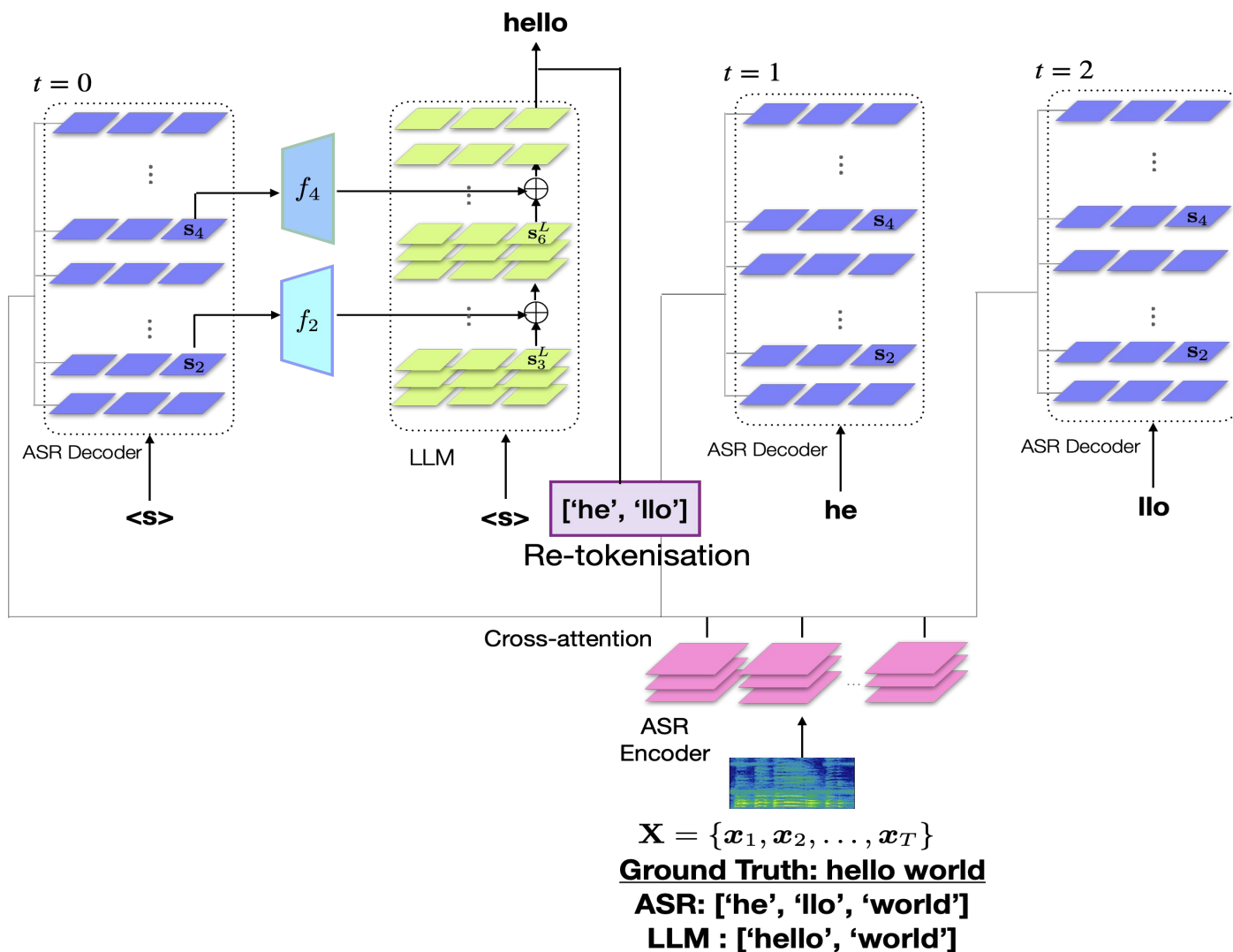


ASR-LLM Aggregation: (Working Example)

- LLM output is re-tokenized with ASR tokenizer

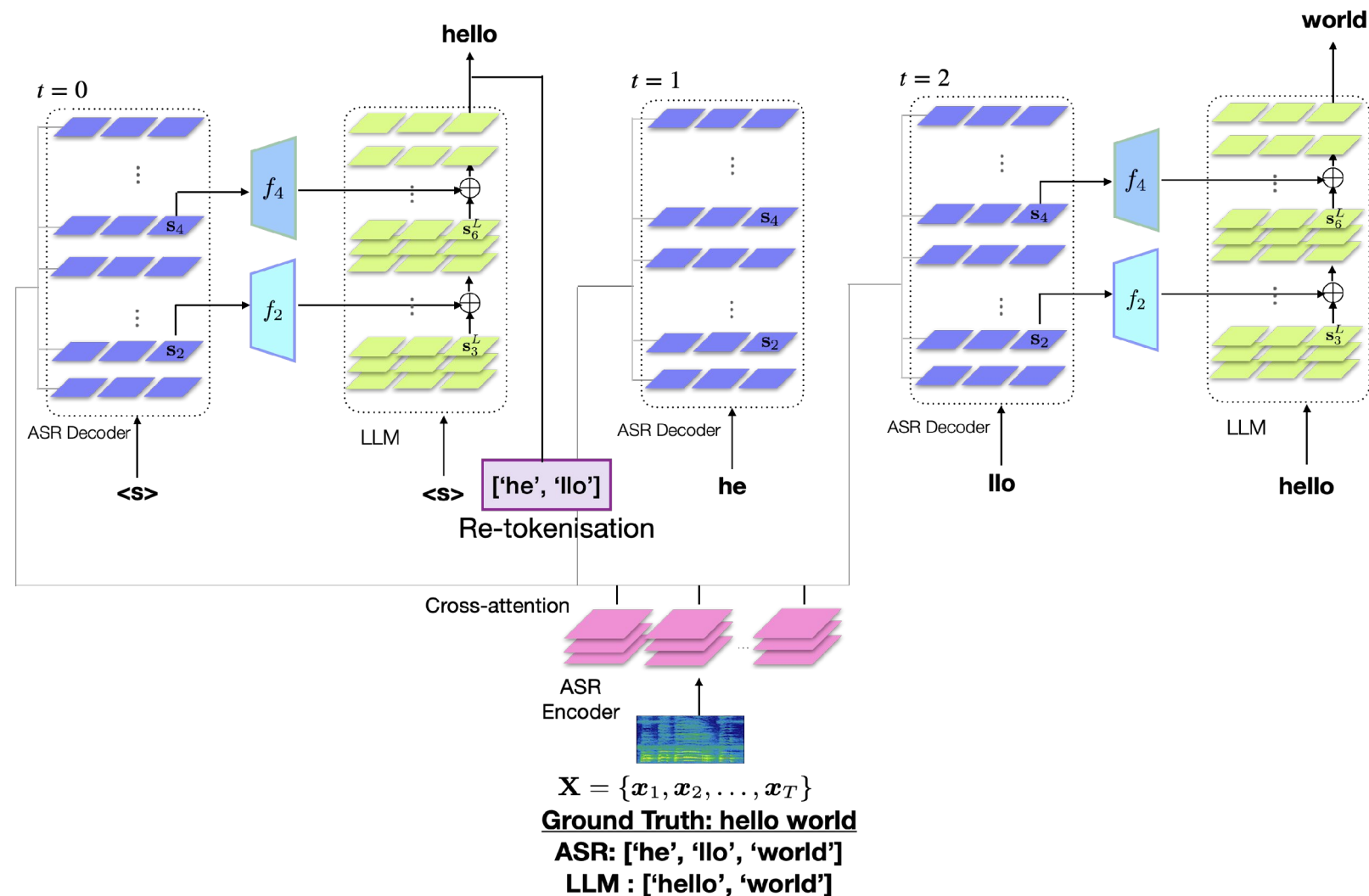


ASR-LLM Aggregation: (Working Example)



- LLM output is re-tokenized with ASR tokenizer
- ASR decoder is advanced with re-tokenized tokens

ASR-LLM Aggregation: (Working Example)

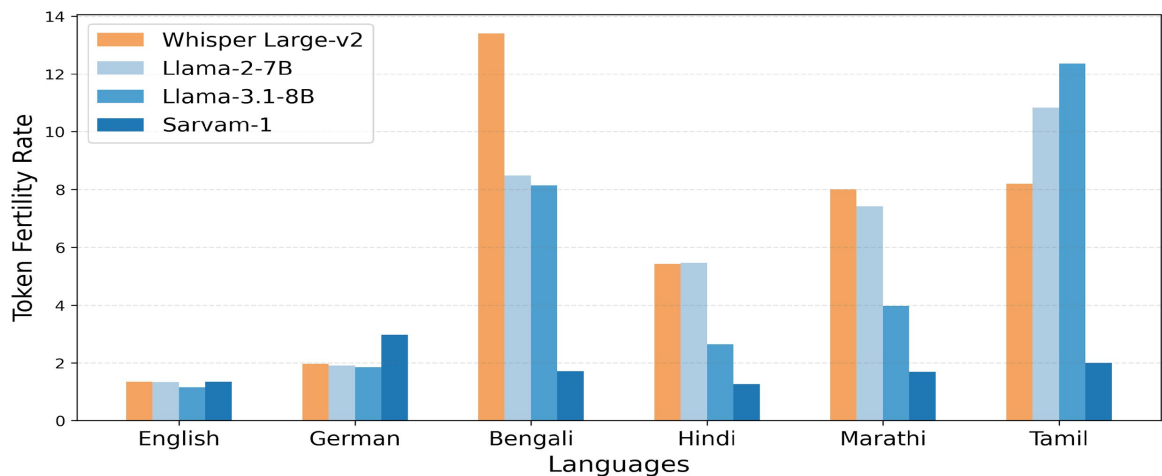


- LLM output is re-tokenized with ASR tokenizer
- ASR decoder is advanced with re-tokenized tokens
- Most recent ASR decoder state is then aggregated by LLM

Challenges with SALSA

Performance of SALSA degrades when the ASR system and the LLM have a large token fertility gap

*Token Fertility → Average number of tokens required for each word



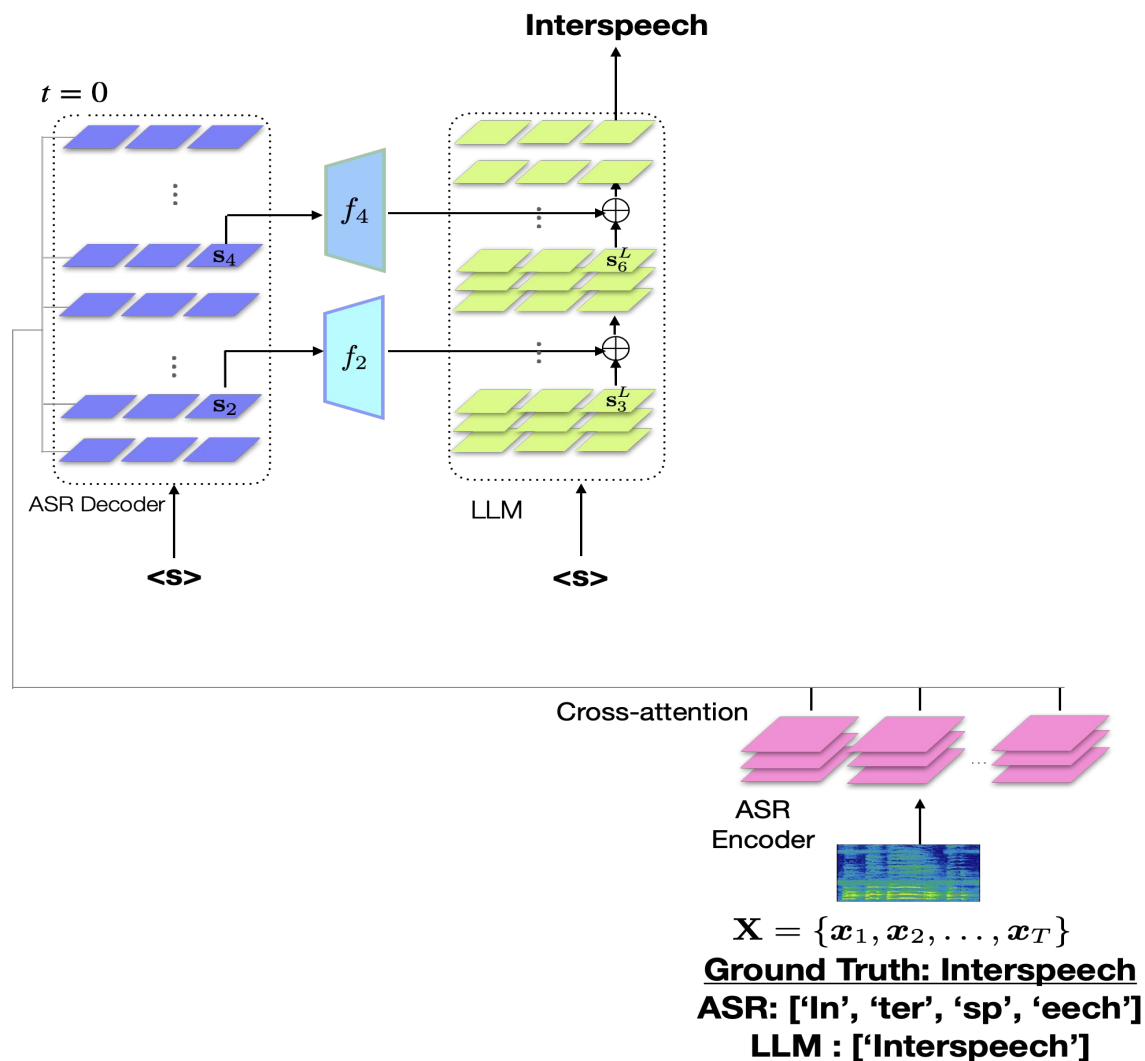
Comparing token fertility rates of Whisper-v2, Llama-2, Llama-3.1, Sarvam-1 across six languages.

Languages	SLAM-ASR LLama3.1-8b	SALSA LLama2.1-7B	SALSA LLama3.1-8b	SALSA Sarvam-1
Hindi	106.6	17.5	24.6	81.4
Marathi	132.4	36.7	46.9	86.0
Tamil	103.2	39.9	42.1	76.0

WER on Fleurs test set with SALSA.

*trained on 10 hours of labelled data

Weakness in SALSA Architecture

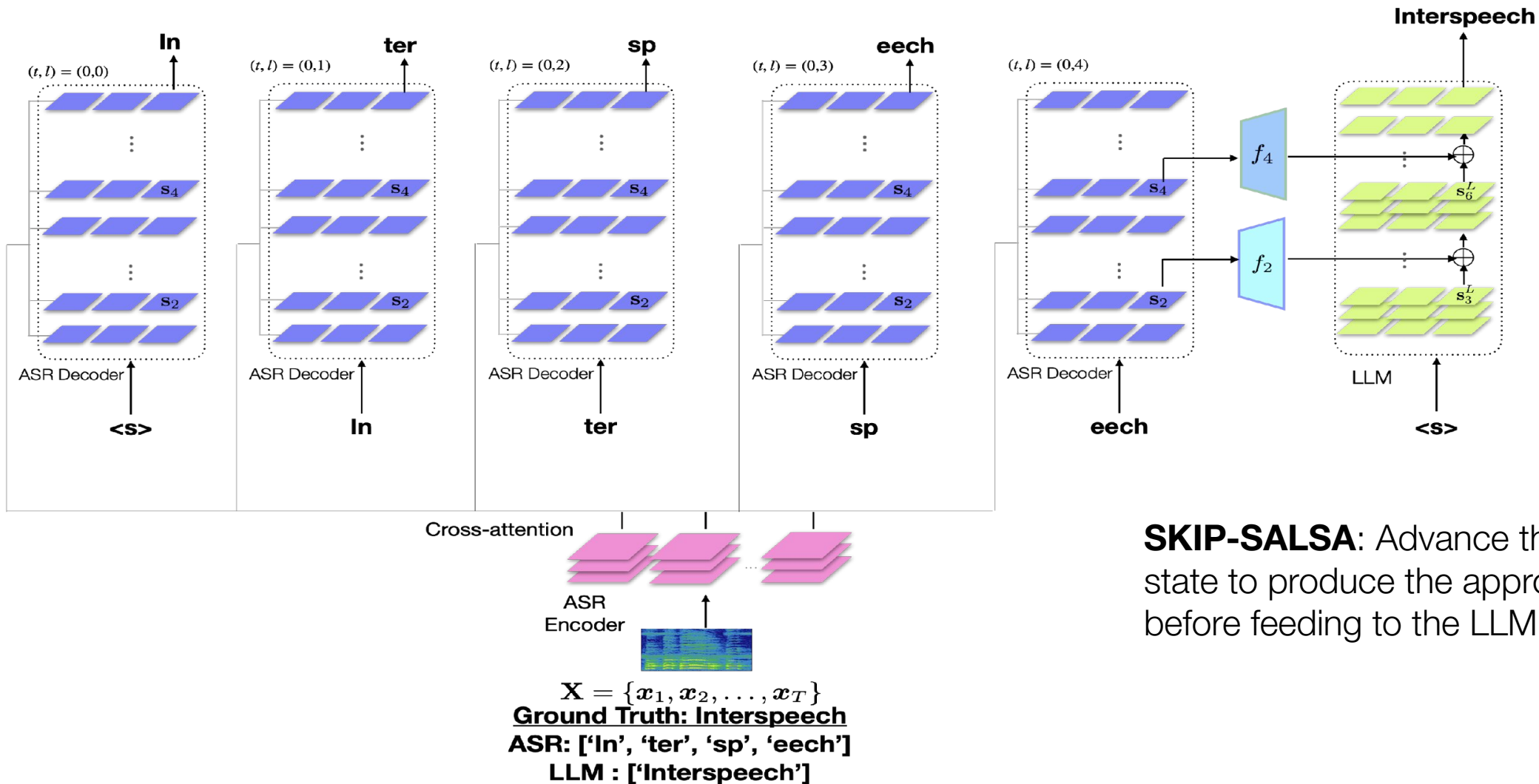


- ASR tokenizes the word Interspeech as ["In", "ter", "sp", "eech"]
- LLM tokenizes the word Interspeech as ["Interspeech"]

Main Issue

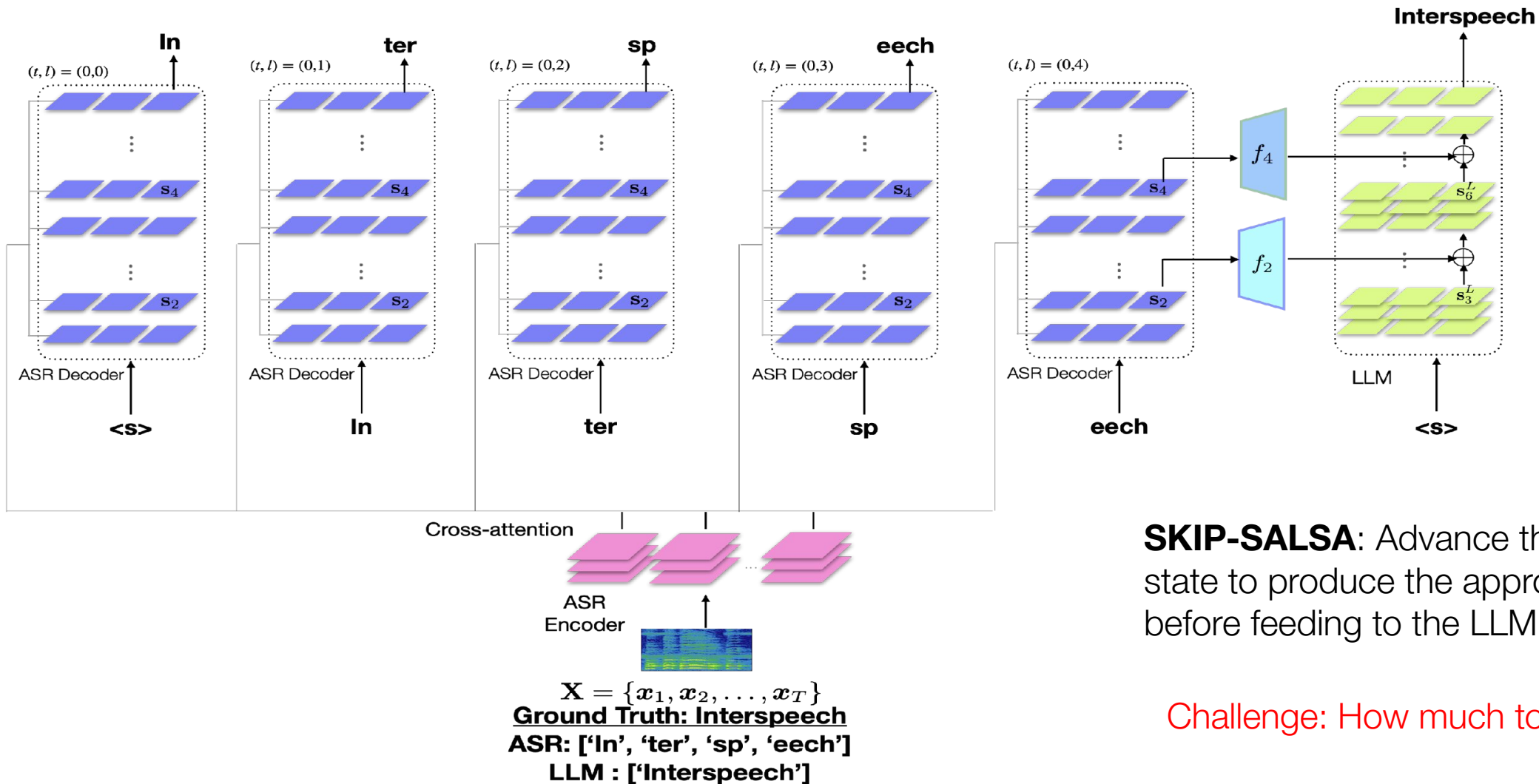
- Insufficient ASR decoder state provided to LLM, especially when they exhibit high token fertility gap
- LLM must rely on the state that produces the word "in" (which takes as input " $\langle s \rangle$ ") to generate the full token

SKIP-SALSA: Skip Synchronous Fusion of ASR LLM Decoders



SKIP-SALSA: Advance the ASR decoder state to produce the appropriate context before feeding to the LLM

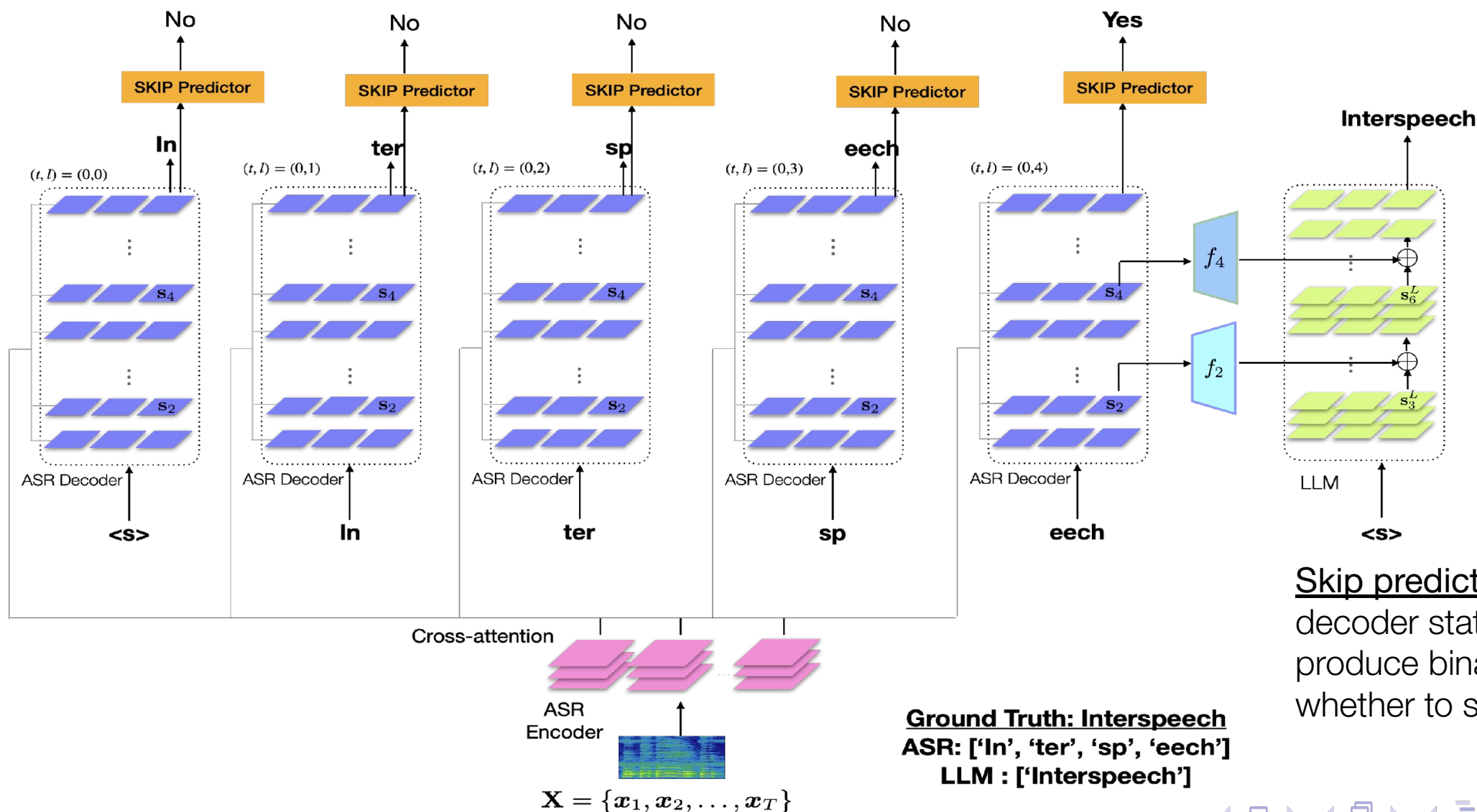
SKIP-SALSA: Skip Synchronous Fusion of ASR LLM Decoders



SKIP-SALSA: Advance the ASR decoder state to produce the appropriate context before feeding to the LLM.

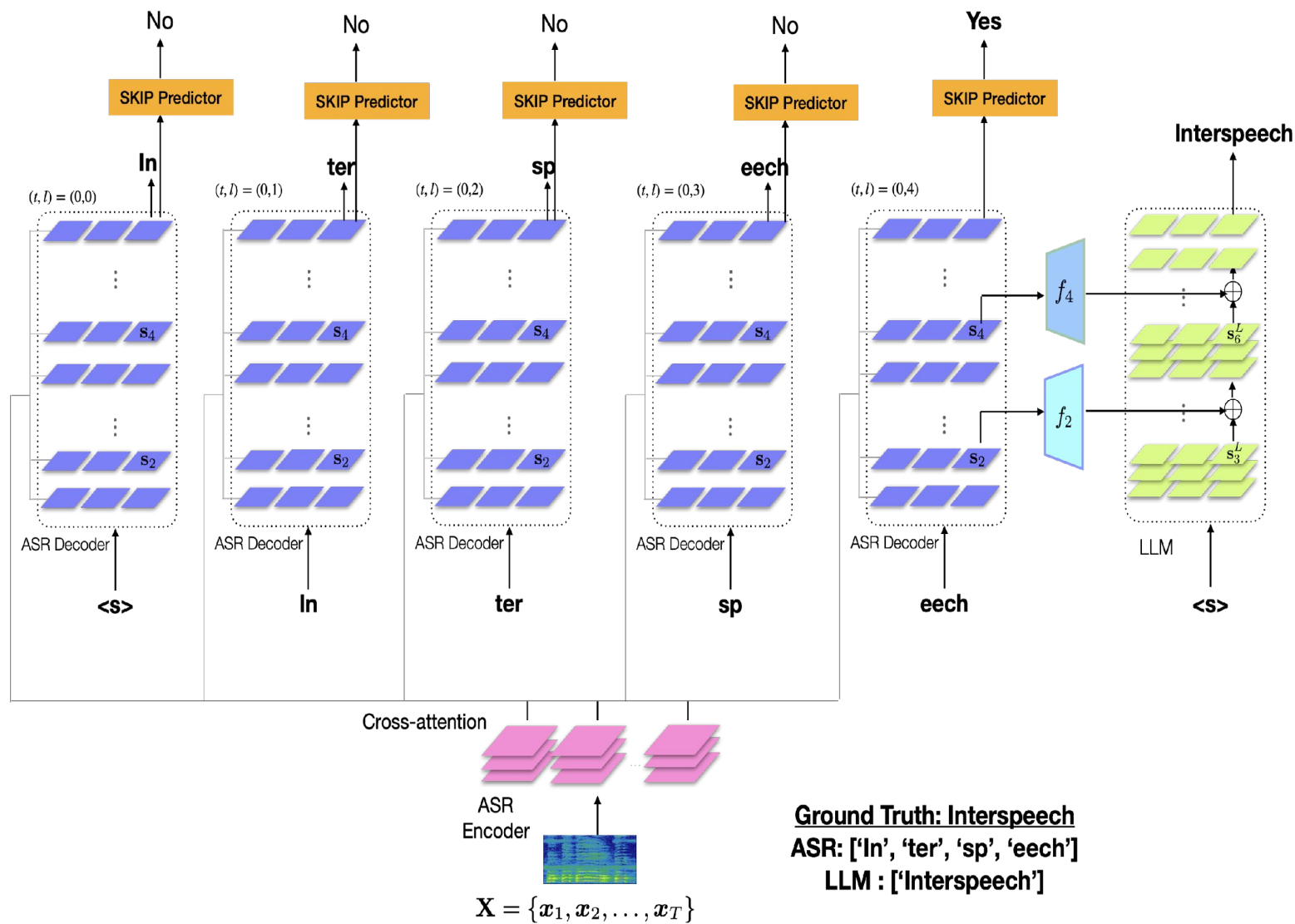
Challenge: How much to advance?

SKIP-SALSA: Skip Synchronous Fusion of ASR LLM Decoders



Skip predictor: Maps the ASR decoder state from last layer to produce binary decision on whether to skip or not?

SKIP-SALSA: Training



- Tokenize the gold transcript using the LLM tokenizer, followed by tokenization via the ASR model
- Coupling strategy utilizing only the advanced decoder state for better context
- During training, the decoder states and skip predictor labels are teacher-forced

SKIP-SALSA: Training

$$\mathcal{J} = \alpha \cdot \mathcal{J}_{\text{ASR}} + \beta \cdot \mathcal{J}_{\text{LLM}} + (1 - \alpha - \beta) \cdot \mathcal{J}_{\text{LP}}$$

- **Trainable components** – Update projection layers, LoRA adapters, skip predictor; freeze ASR & LLM backbones
- **Loss formulation** – Weighted sum of ASR CE loss ($\mathcal{J}_{\text{ASR}}$), LLM CE loss ($\mathcal{J}_{\text{LLM}}$), skip predictor BCE loss ($\mathcal{J}_{\text{LP}}$)

SKIP-SALSA: Training

$$\mathcal{J} = \alpha \cdot \mathcal{J}_{\text{ASR}} + \beta \cdot \mathcal{J}_{\text{LLM}} + (1 - \alpha - \beta) \cdot \mathcal{J}_{\text{LP}}$$

- **Trainable components** – Update projection layers, LoRA adapters, skip predictor; freeze ASR & LLM backbones
- **Loss formulation** – Weighted sum of ASR CE loss ($\mathcal{J}_{\text{ASR}}$), LLM CE loss ($\mathcal{J}_{\text{LLM}}$), skip predictor BCE loss ($\mathcal{J}_{\text{LP}}$)
- **Skip predictor** – MLP on ASR last-layer state; sigmoid-thresholded skip decision; trained with ground-truth alignments

SKIP-SALSA: Inference

Algorithm 1: SKIP-SALSA's decoding algorithm.
Our modifications to SALSA are highlighted in pink.

Input : ASR Model $\mathcal{M}$, LLM Model $\mathcal{L}$, skip
Predictor Model $\mathcal{P}$, Audio: X
Tokenizers $\mathcal{T}$ of ASR, $\mathcal{T}^L$ of LLM
 f_ℓ, F, I : coupling model specification.

Output: Transcript $\{y_0, \dots, y_u\}$

```
1 h: ASR Encoder State ( $X$ )
2 s = ASR Decoder State initialized with SOT token
3  $s^L$  = Initial LLM Decoder State
4  $pi = 0, \mathbf{y} = []$ 
5 for  $i \in \{1 \dots \text{max estimated tokens in } X\}$  do
6    $\mathbf{s}^{skip} = \mathbf{s}, S = 0$ 
7   while  $\mathcal{P}(\mathbf{s}_d^{skip}) < 0.5$  do
8     new_token =  $\mathcal{M}.\text{greedy\_pick}(\mathbf{s}_d^{skip})$ 
9      $\mathbf{s}^{skip} = \mathcal{M}(\mathbf{h}, \mathbf{s}^{skip}, \text{new\_token})$ 
10     $S += 1$ 
11     $\mathcal{M}.\text{reset\_cache}(\text{last\_n} = S)$  # drop last  $S$  states
    from cache
12     $\mathbf{s}_\ell^L = \mathbf{s}_\ell^L + f_\ell(\mathbf{s}_{I(\ell)}^{skip}), \forall \ell \in F$ 
13     $y_i, \mathbf{s}^L = \mathcal{L}(\mathbf{s}^L)$ 
14     $\mathbf{y}.\text{append}(y_i)$ 
15    textY =  $(\mathcal{T}^L)^{-1}(\mathbf{y})$ 
16    if textY ends with a valid utf-8 character then
17      new_tokens =  $\mathcal{T}(\text{textY}[pi:])$ 
18       $\mathbf{s} = \mathcal{M}(\mathbf{h}, \mathbf{s}, \text{new\_tokens})$ 
19       $pi = \text{len}(\text{textY})$ 
20    If  $y_i == \mathcal{T}^L.\text{eos}$  break;
21 return  $\mathbf{y}$ 
```

- Advance ASR states while SKIP predictor active
- Reset ASR KV cache as predictions may not match LLM
- Advance LLM decoder using ASR skip state
- Re-tokenize LLM output to advance ASR states

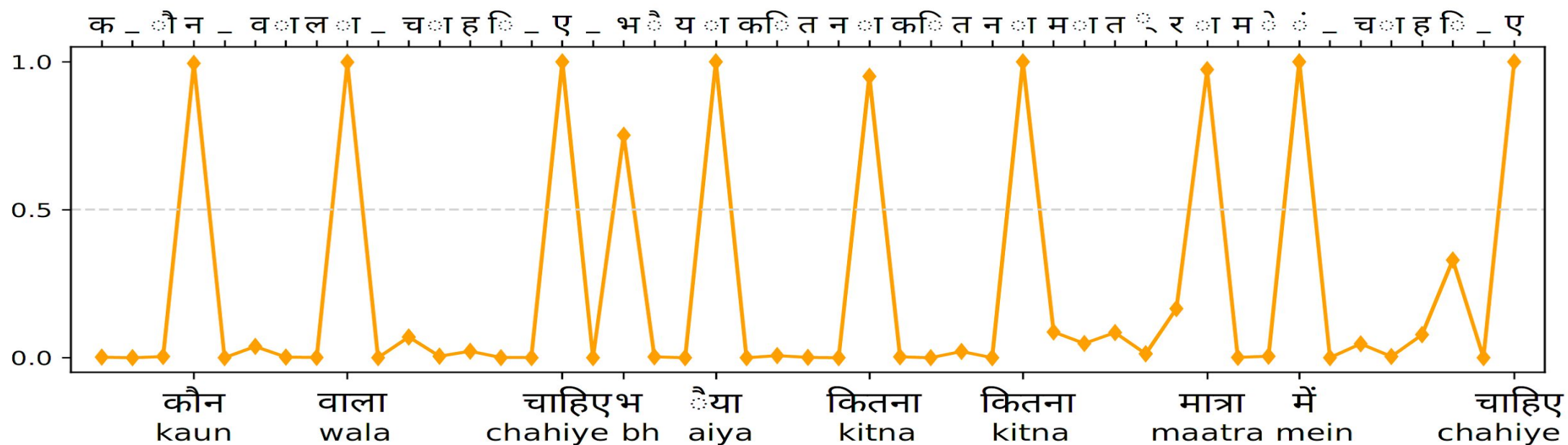
Results

Method	# params	Bengali		Hindi		Marathi		Tamil		Average	
		CER ↓	WER ↓	CER ↓	WER ↓	CER ↓	WER ↓	CER ↓	WER ↓	CER ↓	WER ↓
Our Baselines (Reproduced using official repositories)											
Whisper (Large-v2) [23]	—	105.3	115.5	45.5	60.6	73.0	107.7	48.7	85.3	68.1	92.3
w/ LORA fine-tuning [25]	15M	24.0	43.3	13.9	28.4	20.6	39.5	17.6	44.3	19.0	38.9
SLAM-ASR (w/ Sarvam-1) [17]	17M	23.1	37.6	15.6	27.9	35.1	52.7	19.8	44.2	23.4	40.6
SALSA (w/ Sarvam-1) [21]	11M	26.2	42.6	18.1	31.2	36.9	56.5	20.0	45.1	25.3	43.9
w/ LORA fine-tuned Whisper	26M	22.5	39.5	13.5	26.5	19.9	37.3	15.5	41.5	17.9	36.2
SKIP-SALSA (w/ Sarvam-1)	23M	15.9	31.7	12.8	25.3	18.0	34.8	14.9	39.4	15.4	32.8
w/ LORA fine-tuned Whisper	26M	15.7 †	31.6 †	12.8 †	25.1 †	17.2 †	33.5 †	14.5 †	39.1 †	15.0 †	32.3 †

- Skip-Salsa **significantly improves WERs** compared to Salsa.
- SLAM-ASR struggles with medium-resourced languages compared to SKIP-SALSA.

- ASR: Whisper Large-v2
- LLM: sarvam-1
- Data: ~230 hours of labeled speech in each language

Accuracy of Skip Predictor



Skip predictor achieves an average accuracy between 94.9% and 97.7% respectively, across the chosen languages

Anecdotal Examples

Groundtruth	sharda (ʃɛr d a:) property dealer (d i: l ɹ:) ke naam se dusra office (ɑ: f i s h) hai
SALSA	tarda (tɛr d a:) property realer (r i: l ɹ:) ke naam se dusra afiq (ɑ: f i q) hai
SKIP-SALSA	sharda (ʃɛr d a:) property dealer (d i: l ɹ:) ke naam se dusra office (ɑ: f i s h) hai
Groundtruth	haan sir kaay dadhmadhye (d:ɖhəmədhye) problem aahe kaay sir
SALSA	haan sir kaay dandimadhye (d:ɖɖ:i:mədhye) problem aahe kaay sir
SKIP-SALSA	haan sir kaay dadhmadhye (d:ɖhəmədhye) problem aahe kaay sir
Groundtruth	... parivartanaikku esaipi korikkai (ko:ɹik:ai) vaikkalama (vaik:əla:ma:)
SALSA	... parivartanaikku esaipi koriyirukkavekkilama (ko:ɹijɹuk:əve:k:ila:ma:)
SKIP-SALSA	... parivartanaikku esaipi korikkai (ko:ɹik:ai) vaikkalama (vaik:əla:ma:)

Anecdotes illustrating how **SALSA** suffers because of uninformative decoder state, that **SKIP-SALSA** surmounts this limitation with skip predictions

Impact of SKIP-SALSA on RTF

- Highest RTF of 0.81, with a moderate increase over SLAM-ASR (+44.6%) and SALSA (+26.6%).
- Extra cost stems from additional ASR decoder steps and selective backtracking when states are not adopted by the LLM.
- Improved accuracy from better ASR–LLM synchronization offsets the modest computational overhead.

Conclusions

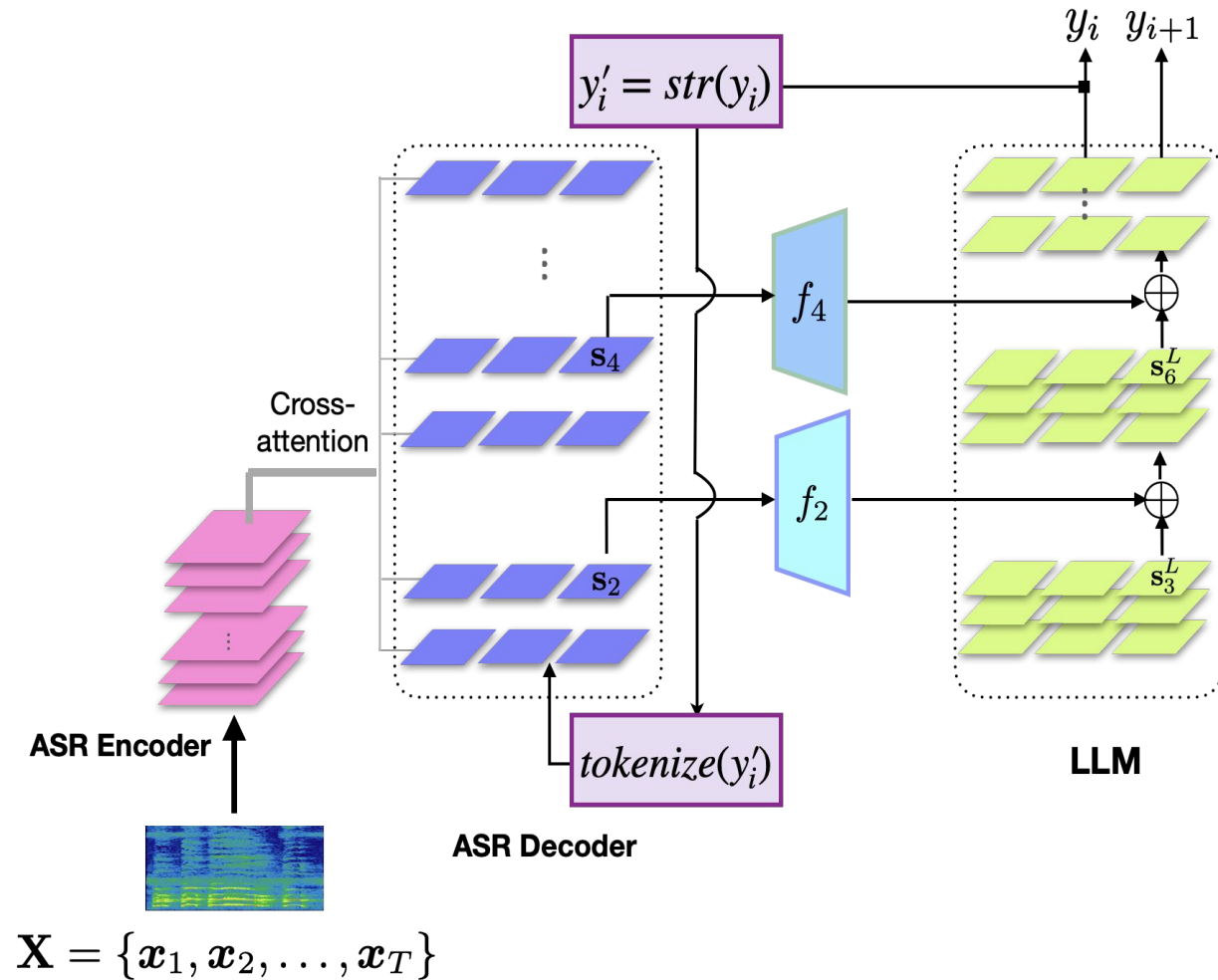
- **Adaptive skip-synchronous decoding:** Allows the LLM to look ahead to future ASR states to handle token fertility gaps
- **Consistent performance gains:** Achieves up to 20% relative WER reduction across languages and resource levels
- **Resource-efficient integration:** Maintains SALSA's lightweight coupling with minimal extra cost

Code: <https://github.com/csalt-research/salsa>



Thank You!

SALSA Training



- Tokenize the gold transcript using the LLM tokenizer, followed by the tokenization via the ASR model
- Standard cross-entropy based training using probabilities from the LLM decoder
- Coupling strategy utilizing only the last decoder states is sufficient since both ASR/LLM decoders are autoregressive